

RASTRO

Transparencia y Datos Abiertos Colombia — Hackathon Croma

ARQUITECTURA DE LA API DE CROMA

Arquitectura de la API de Croma

Protocolo, patrones de respuesta, catálogo de herramientas y capa de normalización de la fuente de datos externa que sostiene a RASTRO.

Versión 1.0 — Agosto de 2026

Proyecto: RASTRO — motor de transparencia en contratación pública colombiana

1. Qué es Croma

Croma es un servidor MCP (Model Context Protocol) que expone registros públicos del Estado colombiano como herramientas invocables — cada fuente oficial (SECOP, RUES, Rama Judicial, Procuraduría, Contraloría, Contaduría, ADRES, SIMIT, Policía Nacional) corresponde a una o más herramientas con nombre y esquema de parámetros propios. RASTRO se conecta a Croma como cliente MCP directo desde el backend — no hay un modelo de lenguaje en el camino de cada consulta; el cliente llama a la herramienta, recibe una respuesta estructurada y la normaliza.

2. Protocolo y transporte

- **Transporte:** MCP sobre HTTPS, vía `StreamableHTTPClientTransport` del SDK oficial (`@modelcontextprotocol/sdk`).
- **Endpoint:** `CROMA_MCP_URL`, por defecto `https://api.croma.run/mcp`.
- **Autenticación:** cabecera `Authorization: Bearer <CROMA_API_KEY>`.
- **Conexión persistente:** el backend mantiene un único cliente MCP conectado durante todo el ciclo de vida del proceso, reutilizado entre consultas — no se reconecta en cada llamada.
- **Timeout por llamada:** 30 segundos. Suficiente margen para fuentes lentas de alto volumen sin bloquear una consulta indefinidamente.

3. Patrones de respuesta

3.1 Respuesta síncrona

La mayoría de las herramientas responden de inmediato con `{ status: "completed", data: {...} }`.

3.2 Trabajo asíncrono (fuentes lentas)

Algunas fuentes —Contaduría es el caso observado en producción— responden primero con `{ status: "pending", job_id, status_url }`. El cliente sondea `status_url` cada 1.5 segundos, hasta 20 intentos, hasta recibir `completed` o `failed`.

3.3 "No encontrado" como respuesta válida

Cuando el sujeto consultado no tiene registro en una fuente, Croma responde `found: false` de forma explícita — es una respuesta definitiva, no un error ni una ausencia de dato. RASTRO la trata como tal: se clasifica como "sin hallazgo", nunca se reintenta ni se oculta.

3.4 Límite de tasa

Croma aplica límite de tasa por organización. El error observado directamente durante el desarrollo tiene esta forma exacta:

"Rate limit exceeded for this organization. Try again in 13645 seconds."

RASTRO no reintenta automáticamente ante este error — lo trata como cualquier otra falla del adaptador primario y degrada de inmediato a los datos de respaldo (ver sección 4), para que un límite de tasa nunca bloquee una consulta ni deje al usuario esperando.

4. Resiliencia del lado de RASTRO

Dos capas de recuperación ante fallas, en orden:

- **Reintento de sesión** (`CromaAdapter._call`): si la llamada falla por un problema de conexión (la sesión MCP persistente puede caer por inactividad o un corte de red y quedar "viva" pero rota), se descarta el cliente cacheado y se reintenta una vez con una conexión nueva antes de propagar el error.
- **Degradación a datos de respaldo** (`dataSource.js`): si el reintento también falla —incluye el caso de límite de tasa—, la llamada cae de forma transparente a `MockAdapter`, que implementa el mismo contrato. El servicio de negocio que la invocó nunca se entera de cuál de los dos adaptadores respondió.

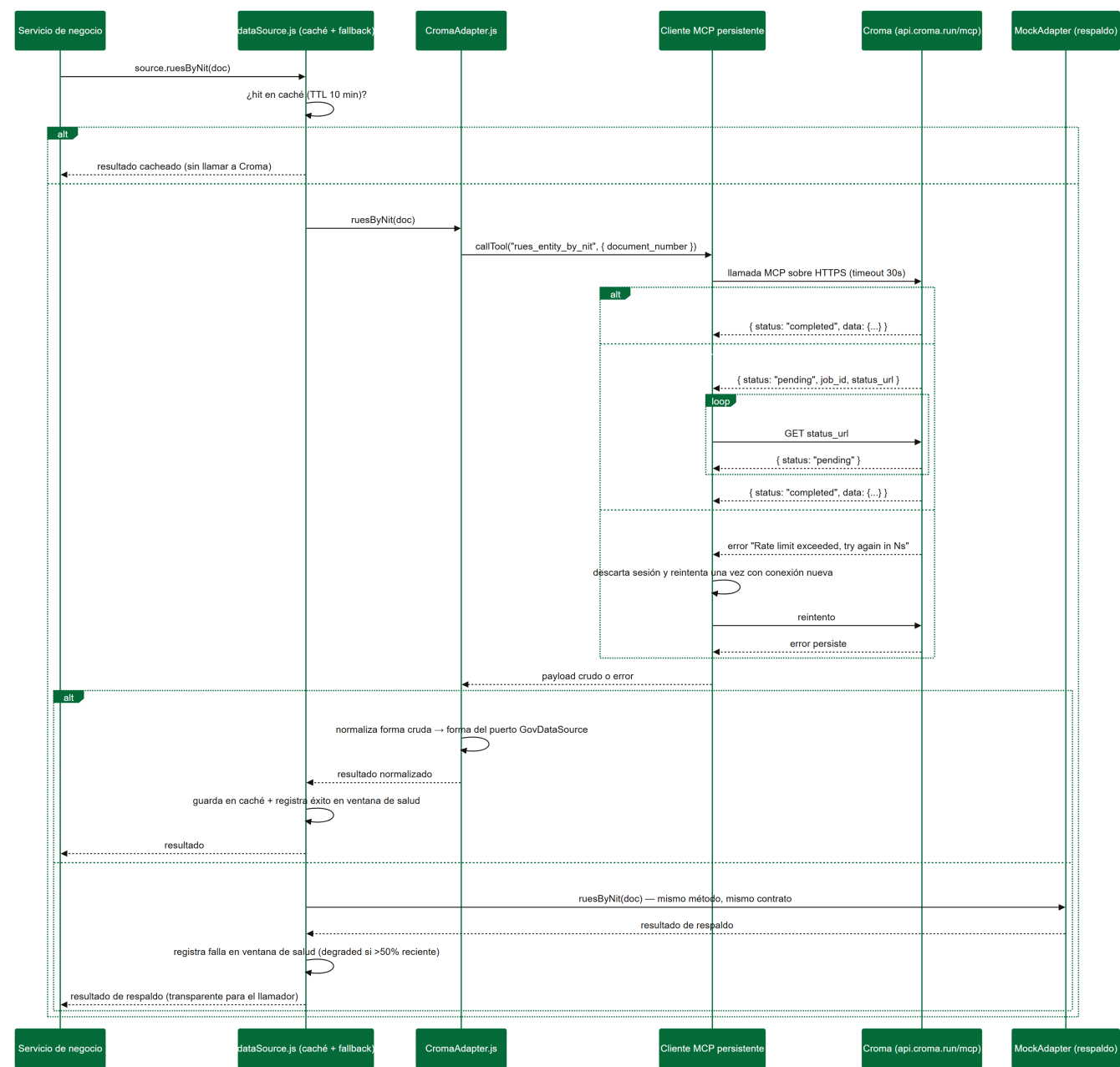


Figura 1 — Ciclo de vida de una llamada a Croma, incluyendo trabajo asíncrono y degradación por límite de tasa.

5. Catálogo de herramientas usadas por RASTRO

Croma expone decenas de herramientas — incluye fuentes de Perú y México, y varias fuentes colombianas adicionales (DIAN, Consejo de Estado, SCJN, RUNT, entre otras) que RASTRO no consulta por estar fuera de su alcance de producto. Esta es la lista de las catorce herramientas que sí usa, todas mapeadas en `src/adapters/CromaAdapter.js`.

Herramienta Croma	Fuente oficial que representa	Uso y notas relevantes en RASTRO
<code>rues_entity_by_nit</code>	RUES	Resolver un NIT: registro mercantil, estado, representantes, indicadores financieros.
<code>rues_entities_by_name</code>	RUES	Búsqueda por nombre, paginada — alimenta la cascada de normalización de <code>search.js</code> .
<code>secop_processes_by_entity</code>	SECOP II	Listado liviano de procesos de una entidad contratante. No trae proveedor ni valor adjudicado.
<code>secop_process</code>	SECOP II	Detalle de un proceso: adjudicaciones reales (proveedor, valor, indicador de consorcio/UT). Única fuente de este dato.
<code>secop_contract</code>	SECOP II	Seguimiento financiero de un contrato: valor, facturado, pagado, adiciones, garantías, plan de entrega.
<code>secop_contracts_by_provider</code>	SECOP II	Historial de contratos ganados por un proveedor específico.
<code>secop_sanctions_by_provider</code>	SECOP II	Sanciones contractuales registradas contra un proveedor.
<code>procuraduria_disciplinary_records</code>	Procuraduría (SIRI)	Antecedentes disciplinarios.
<code>contraloria_fiscal_records</code>	Contraloría (SIBOR)	Responsabilidad fiscal. Certifica solo personas naturales — para un NIT se marca "no aplica".
<code>rama_judicial_cases_by_entity</code>	Rama Judicial (CPNU)	Procesos judiciales activos. Requiere nombre completo, no solo el número de documento.
<code>contaduria_state_delinquent_debtors</code>	Contaduría (Boletín de Deudores Morosos)	Nunca responde <code>found:false</code> — certifica un veredicto explícito para dos leyes distintas (901/2004 y 1066/2006).
<code>policia_criminal_records</code>	Policía Nacional	Antecedentes penales.
<code>adres_affiliation_status</code>	ADRES (BDUA)	Afiliación vigente al sistema de salud.
<code>simit_account_status</code>	SIMIT	Multas de tránsito pendientes y acuerdos de pago.

6. Capa de normalización — CromaAdapter.js

Cada método de `CromaAdapter` cumple el mismo contrato: recibe los argumentos que define el puerto `GovDataSource`, invoca la herramienta Croma correspondiente, y traduce la forma cruda de esa herramienta

(nombres de campo en inglés y snake_case, estructuras propias de cada fuente) a la forma estable que el resto de RASTRO consume. Esta capa es la única parte del sistema que conoce el formato real de Croma — si Croma cambia un nombre de campo o el proveedor de datos cambia por completo, el ajuste ocurre aquí, sin tocar scoreEngine.js, concentration.js ni ningún otro servicio de negocio.

7. Límite estructural descubierto: consorcios y uniones temporales

Durante el desarrollo se investigó directamente si Croma expone la composición de un consorcio o unión temporal como dato estructurado — necesario para construir un mapa de relaciones entre integrantes. El hallazgo, verificado en producción:

- `secop_process` marca `is_group`: true cuando el ganador de un contrato es un consorcio o unión temporal, y entrega su nombre comercial completo en texto libre — por ejemplo *"UNION TEMPORAL CALI 2024-SBS-SOLIDARIA-CHUBB-MAPFRE-PREVISORA"*.
- Ese nombre no viene acompañado de una lista estructurada de NIT por integrante.
- Se buscó esa misma unión temporal por nombre en `rues_entities_by_name`: cero resultados. Confirma que, en Colombia, un consorcio o unión temporal no es persona jurídica propia y por lo tanto no se registra en cámara de comercio con sus integrantes listados — no es una limitación de Croma, es una característica del marco legal que representa.

Por esta razón, `concentration.js` usa el proveedor y el valor real de cada adjudicación (el dato que sí es estructurado y confiable) en vez de intentar reconstruir la composición de un consorcio a partir de su nombre en texto libre — que produciría un dato adivinado, no verificado, en contradicción directa con el principio de evidencia trazable del producto.